

Work Breakdown Structure (WBS)

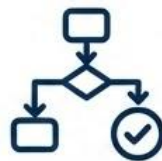
เครื่องมือสำคัญในการจัดการโครงการ

วัตถุประสงค์การเรียนรู้

เมื่อจบบทเรียนนี้ คุณจะสามารภ:



อธิบายความหมายและ
ความสำคัญของ WBS



เข้าใจหลักการและกฎ
เกณฑ์ในการสร้าง WBS



สร้าง WBS สำหรับ
โครงการ Software ได้



ประยุกต์ใช้ WBS ในการ
วางแผนโครงการจริง

ทำไมโครงการถึงล้มเหลว? 🤔



❌ ไม่รู้ว่ามีงานอะไรบ้าง
(Scope Creep)



❌ ประมาณเวลาและ
งบประมาณผิดพลาด



❌ มอบหมายงาน
ไม่ชัดเจน ทีมสับสน



❌ ลืมงานสำคัญ
เช่น Testing,
Documentation



❌ ติดตามความ
คืบหน้าไม่ได้

ตัวอย่างจริง: โครงการที่ล้มเหลว

โครงการพัฒนาแอปสั่งอาหาร

Timeline: 3 เดือน | งบประมาณ: 500,000 บาท



ใช้เวลา 8 เดือน
(เกิน 166%)



ใช้งบ 1,200,000 บาท
(เกิน 140%)



ลูกค้าไม่พอใจ
ต้องแก้ไขหลาย Round



สาเหตุหลัก: ไม่มีการแบ่งงานชัดเจน

WBS คืออะไร?

Work Breakdown Structure (WBS)

- 🧩 การแบ่งโครงการออกเป็นส่วนย่อยๆ
- 🏗️ ในลักษณะลำดับชั้น (Hierarchical)
- ✅ เพื่อให้จัดการได้ง่ายขึ้น

“แบ่งงานใหญ่ให้เป็นงานเล็กจนกระทั่งจัดการได้”



WBS เหมือนกับ...



สารบัญของหนังสือ

บทที่ 1

- 1.1 หัวข้อย่อย
- 1.2 หัวข้อย่อย



โครงสร้างโฟลเดอร์

Project/

- Design/
- Development/

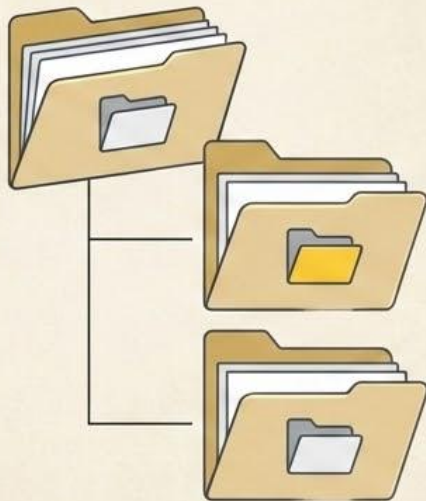


สูตรทำพิซซ่า

- เตรียมแป้ง
- ทำซอส
- อบ



โครงสร้างโฟลเดอร์



สูตรทำพิซซ่า



เตรียมแป้ง

- ...
- ...



ทำซอส

- ...
- ...

อบ

- ...
- ...



ทำไมต้องใช้ WBS? ✨

1



มองเห็นภาพรวม
โครงการทั้งหมด

2



ประมาณการเวลา
และต้นทุนแม่นยำ

3



มอบหมายงานชัดเจน
รู้ว่าใครทำอะไร

4



ติดตามความ
คืบหน้าได้ง่าย

5



ลดความเสี่ยง
จากการมองข้ามงาน

6



สื่อสารกับทีมและ
Stakeholder ได้ดี

องค์ประกอบของ WBS

โครงสร้าง WBS ประกอบด้วย:



Level 0: โครงการทั้งหมด (Project)



Level 1: งานหลัก (Major Deliverables)



Level 2: งานรอง (Sub-deliverables)



Level 3: Work Package (งานที่มอบหมายได้)

Work Package คือ:

- ✓ สามารถประมาณเวลา/ค่าใช้จ่ายได้
- ✓ มอบหมายให้คนหรือทีมได้
- ✓ มี Deliverable ที่วัดผลได้

โครงการ: "ทำรายงานกลุ่ม"

ทำรายงานวิชา Software Engineering



1. เตรียมการ



1.1 ประชุมแบ่งหน้าที่



1.2 รวบรวมข้อมูล



2. เขียนเนื้อหา



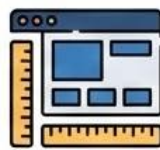
2.1 เขียนบทนำ



2.2 เขียนเนื้อหาหลัก



2.3 เขียนสรุป



3. จัดรูปแบบ



3.1 ออกแบบ Layout



3.2 ตรวจสอบ Format



4. ส่งมอบ

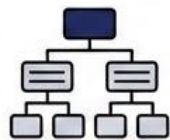


4.1 Proofread



4.2 ส่งอาจารย์

WBS ≠ To-Do List ≠ Gantt Chart



WBS



โครงสร้างลำดับชั้น



What (อะไร)



ไม่มีเวลา



แบ่งเป็นชั้นๆ



To-Do List



รายการแบบ



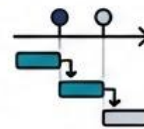
What (อะไร)



อาจมี Due Date



ไม่มีลำดับชั้น



Gantt
Chart



Timeline



When (เมื่อไหร่)



มี Duration



แสดงบนแกนเวลา



WBS = รากฐาน → นำไปสร้าง Schedule ได้



PART 2
PRINCIPLES & RULES

ประเภทของ WBS



① Phase-based

Planning → Design →
Development → Testing



② Deliverable-based

User Module → Payment
Module → Report



③ Hybrid

Phase + Deliverable

Phase-based WBS

เหมาะกับ: Waterfall, Traditional Projects

โครงการพัฒนาเว็บไซต์ E-Commerce



Phase 1:
Planning & Analysis



Phase 2:
Design



Phase 3:
Development



Phase 4:
Testing



Phase 5:
Deployment



Phase 6:
Maintenance

ข้อดี

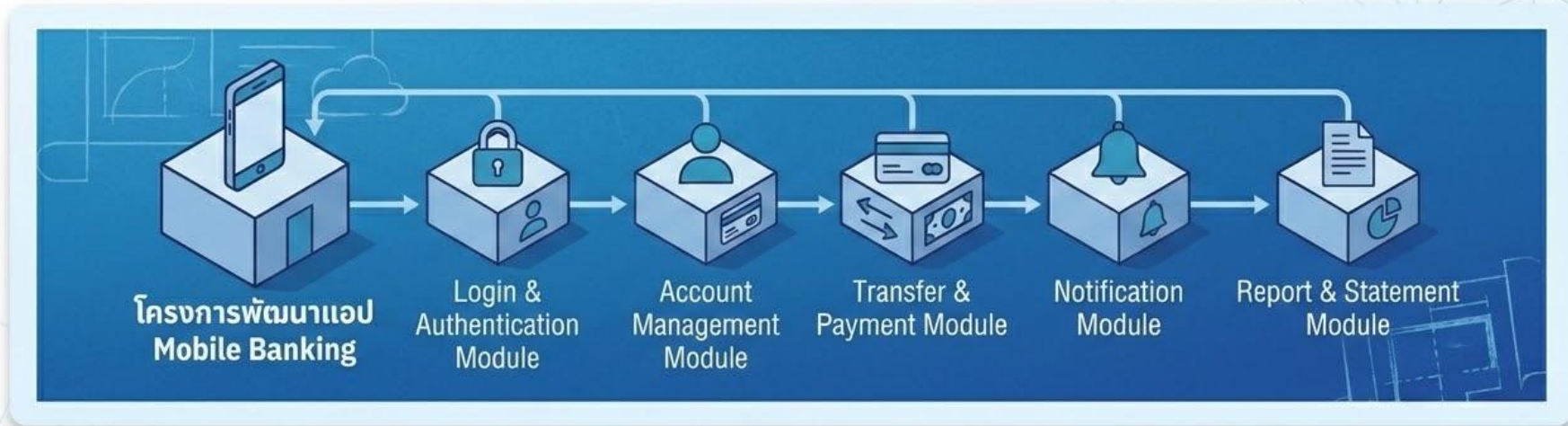
- ✓ เห็น Flow ของโครงการชัดเจน
- ✓ ง่ายต่อการติดตาม Progress

ข้อเสีย

- ✗ ยืดหยุ่นน้อย
- ✗ ไม่เหมาะกับ Agile

Deliverable-based WBS

เหมาะกับ: *Feature-driven, Agile Projects*



ข้อดี

- ✓ มุ่งเน้นผลลัพธ์
- ✓ ยืดหยุ่น ทำ Parallel ได้

ข้อเสีย

- ✗ อาจมองข้าม Infrastructure/DevOps

Hybrid WBS

ผสมจุดเด่นของทั้ง 2 แบบ



เหมาะกับโครงการขนาดใหญ่ที่มีหลาย Phase

กฎสำคัญ #1 - The 100% Rule



“WBS ต้องครอบคลุมงานทั้งหมด 100%
ของโครงการ ไม่มากไม่น้อย”

✓ ถูกต้อง:

งานทั้งหมดรวมกัน
= 100% ของ
โครงการ



✗ ผิด:

- ลืมงาน Testing ($< 100\%$)
- ใส่งานที่ไม่เกี่ยวข้อง ($> 100\%$)



Missing Work



Extra Work

กฎสำคัญ #2 - The 8/80 Rule

Work Package แต่ละงานควร:

- มากกว่า 8 ชั่วโมง (1 วัน) • น้อยกว่า 80 ชั่วโมง (2 สัปดาห์)

✗ **น้อยเกินไป** (< 8 ชม.)



“เขียนฟังก์ชัน Login”
(2 ชม.) → รวมกับงานอื่น

✓ **พอดี** (8-80 ชม.)



“พัฒนา User Authentication
API” (40 ชม.)

✗ **มากเกินไป** (> 80 ชม.)

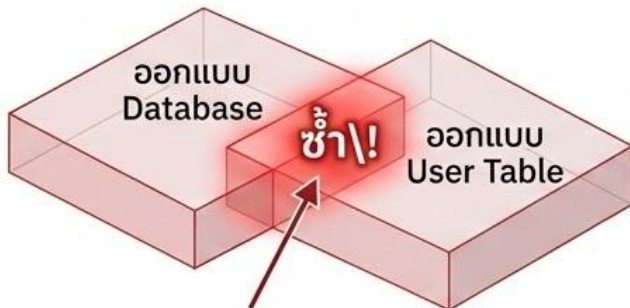


“พัฒนา Backend ทั้งหมด”
(200 ชม.) → แบ่งย่อยต่อ

กฎสำคัญ #3 - Mutually Exclusive 🚫

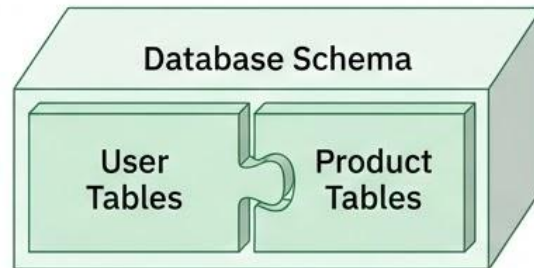
งานแต่ละงานต้องแยกจากกันชัดเจน ไม่มีส่วนที่ทับซ้อนกัน

❌ ผิด (Wrong)



- 'ออกแบบ Database'
- 'ออกแบบ User Table' ← ซ้ำ!

✅ ถูก (Correct)



- 'Database Schema'
 - User Tables
 - Product Tables

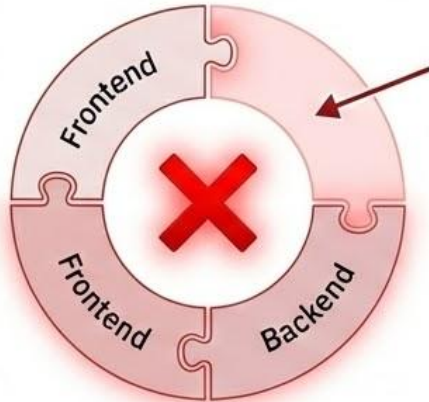


ถ้าทำงาน A เสร็จ ไม่ควรทำซ้ำใน งาน B

กฎสำคัญ #4 - Collectively Exhaustive ✓

งานย่อยทั้งหมดรวมกัน = งานหลัก 100%

✗ ผิด - ขาดหาย (Wrong - Incomplete)

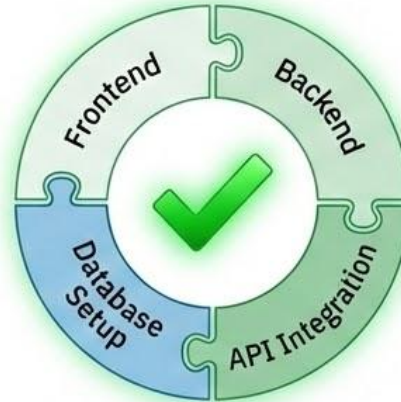


Missing!
(ขาดหาย!)
[ลืมน Integration!]

Development Phase:

- Frontend
- Backend
- [ลืมน Integration!]

✓ ถูก - ครบถ้วน (Right - Complete)



Development Phase:

- Frontend
- Backend
- API Integration
- Database Setup



Check: ถ้าทำงานย่อยครบทุกงาน = งานหลักเสร็จสมบูรณ์หรือไม่?

การตั้งชื่อ Work Package

Best Practices การตั้งชื่อ:

❌ ผิด (Wrong)

- ❌ ทำ API
- ❌ Login
- ❌ งานทดสอบ
- ❌ แก้งบั๊ก



✅ ถูก (Correct)

- ✅ เฉพาะเจาะจง: 'Develop User Login API with JWT'
- ✅ วัดผลได้: 'Complete 10 Unit Tests for Payment Module'



ชื่อที่ดีต้องชัดเจน มีขอบเขต และวัดผลได้

PART 3
LIVE DEMO

สาริต: พัฒนา Mobile App สั่งอาหาร



Timeline: 4 เดือน

ระยะเวลาโครงการ



Team: 5 คน

2 Dev, 1 Designer, 1 Tester, 1 PM



Platform: iOS + Android

แพลตฟอร์มที่รองรับ

Features หลัก: Browse menu, Order, Payment, Tracking



Browse menu



Order



Payment



Tracking

ฟีเจอร์สำคัญ



เราจะสร้าง WBS แบบ Hybrid (Phase + Feature-based)

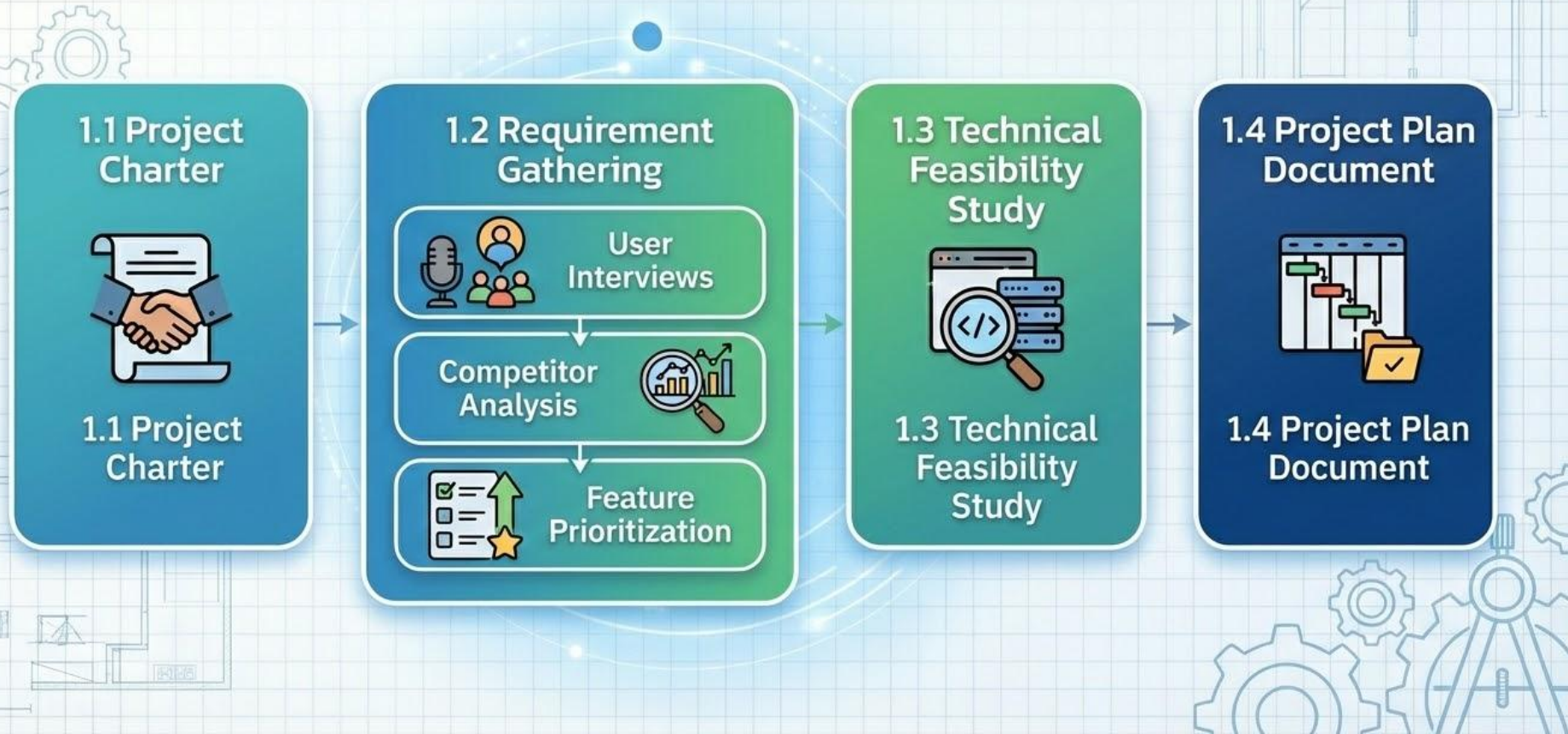
Level 1: Major Deliverables

พัฒนาแอปสั่งอาหาร



คิดว่าครบไหม? มีอะไรที่ลืมไหม?

1. Project Initiation & Planning

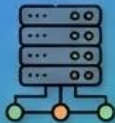


2. Design



2.1 UI/UX Design

-  Wireframes
-  Mockups
-  Prototype



2.2 System Architecture Design

-  Frontend Architecture
-  Backend Architecture
-  Database Design



2.3 API Design Document



Deliverables:

- ✓ Design Specifications
- ✓ Architecture Diagram
- ✓ API Documentation

3. Core Features Development



3.1 User Management

-  Registration & Login
-  Profile
-  Auth (JWT)



3.2 Restaurant & Menu Module

-  Restaurant Listing
-  Menu Display
-  Search



3.3 Order Management

-  Cart
-  Order Placement
-  Order History



3.4 Payment Integration

-  Payment Gateway
-  Transaction Management



3.5 Delivery Tracking

-  Real-time Tracking
-  Notification

4. Testing & QA



4.1 Unit Testing

-  Frontend Unit Tests
-  Backend Unit Tests



4.2 Integration Testing

-  API Integration Tests
-  Payment Integration Tests



4.3 System Testing

-  Functional
-  Performance
-  Security



4.4 User Acceptance Testing (UAT)

- ✓ User feedback & approval



4.5 Bug Fixing & Regression Testing

- ✓ Issue resolution & re-testing

✓ ห้ามลืม Testing!

5. Deployment



5.1 Environment Setup

-  Production Server
-  Database Migration
-  CDN Configuration



5.2 App Store Submission

-  iOS App Store
-  Google Play Store



5.3 Beta Testing (Soft Launch)

- Controlled user group testing



5.4 Official Launch

- Public release & marketing



5.5 Post-launch Monitoring

- Performance, feedback & analytics

6. Project Management (ตลอดโครงการ)



6.1 Sprint Planning (Agile)



6.2 Daily Standups



6.3 Risk Management



6.5 Stakeholder Communication



6.6 Documentation



6.4 Quality Assurance



- Technical Documentation
- User Manual
- API Documentation



งานที่ไม่ใช่ Development แต่สำคัญมาก!

WBS สมบูรณ์ - ภาพรวม

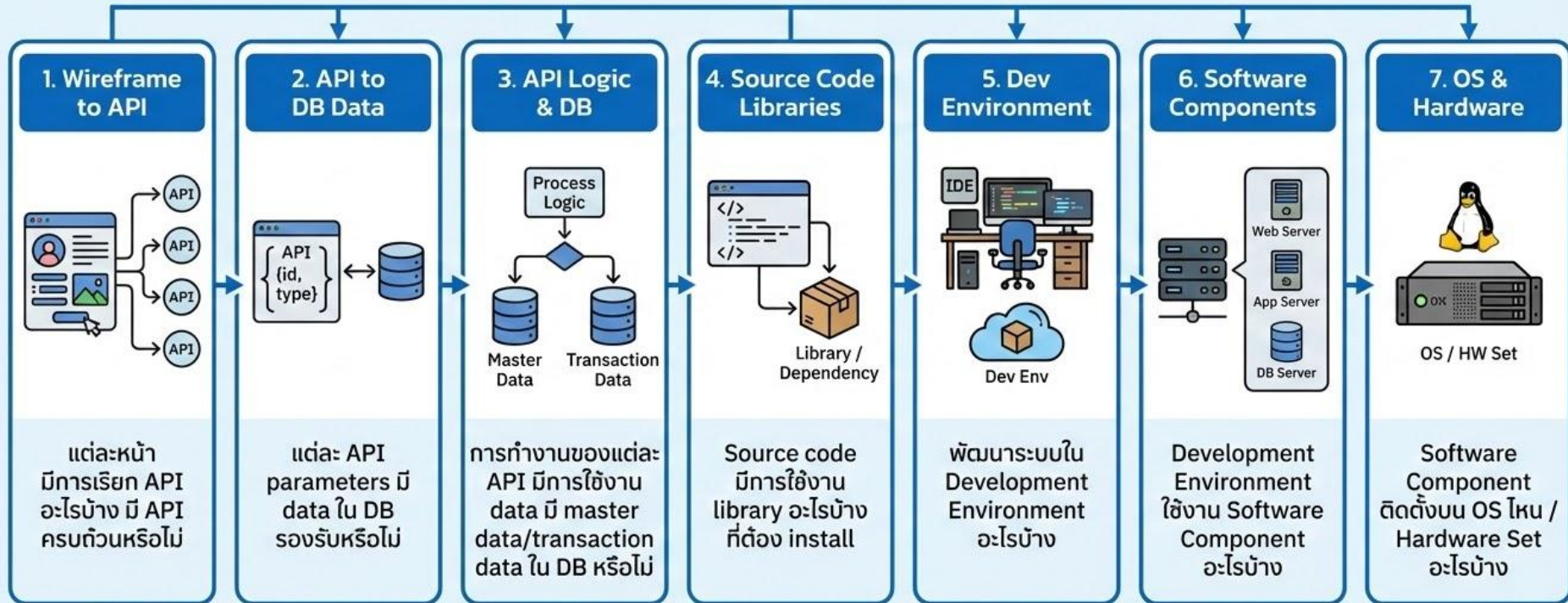
พัฒนาแอปสั่งอาหาร



**รวม: 6 Major Deliverables,
38+ Work Packages**

การตรวจสอบความสมบูรณ์ของ WBS

ตรวจสอบ Alignment แต่ละ layer ต่อไปนี้



รายละเอียดต่างๆ แสดงใน System Diagram / System Architecture

PART 4

COMMON MISTAKES

✗ WBS ที่ไม่ดี #1: ละเอียดเกินไป

ปัญหา: Micromanagement

Development > Login Page > Frontend

- ─ สร้างไฟล์ login.html (1 ชม.) ✗
- ─ เขียน CSS สำหรับปุ่ม Login (30 นาที) ✗
- ─ เขียน JavaScript validation (2 ชม.) ✗
- ─ ทดสอบบน Chrome (30 นาที) ✗
- ─ ทดสอบบน Firefox (30 นาที) ✗



✓ แก๊ซ #1: รวมงานให้เหมาะสม

User Authentication Module (40 ชม.)

Login Page
Development
(8 ชม.)

Registration Page
(8 ชม.)

Password Reset
Feature
(6 ชม.)

- 📄 HTML/CSS/JS
- 📋 Form Validation
- ✓ Cross-browser Testing

Session
Management
(8 ชม.)



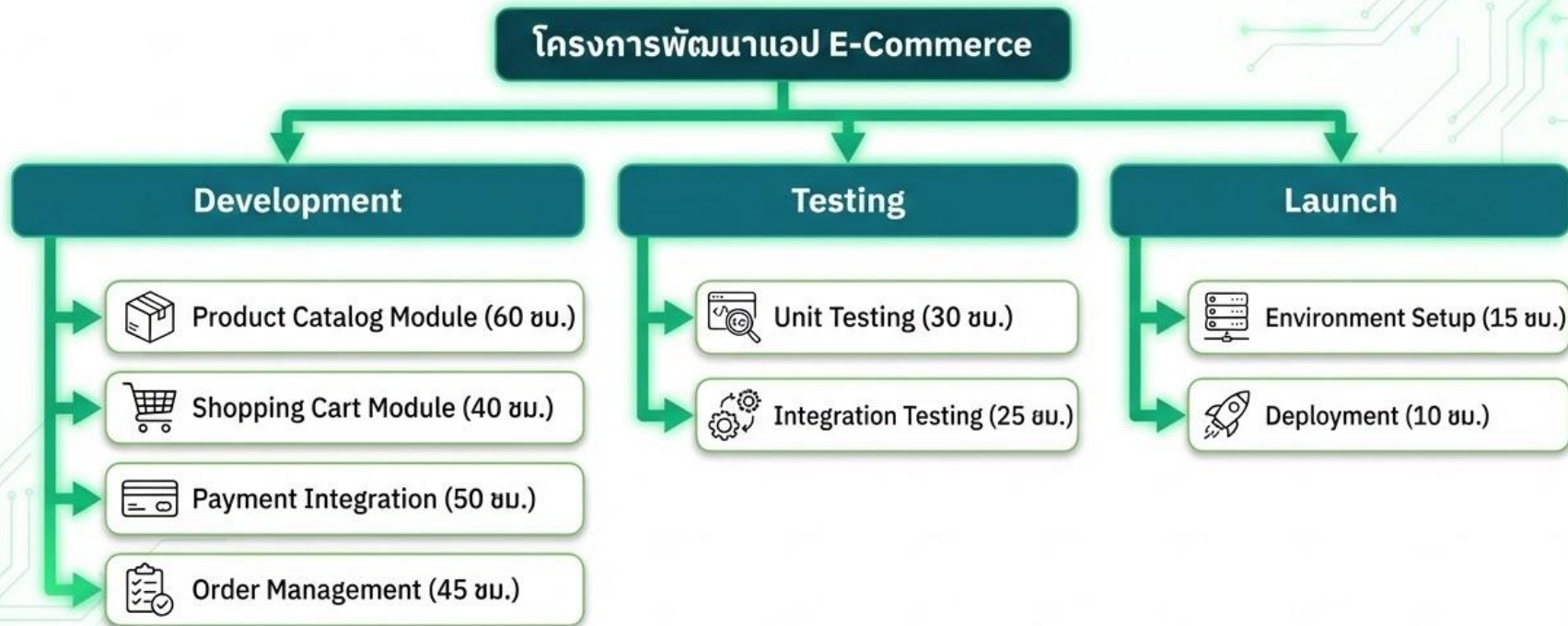
❌ WBS ที่ไม่ดี #2: หยาดเกินไป

ปัญหา: งานใหญ่เกิน ประสิทธิภาพไม่ได้





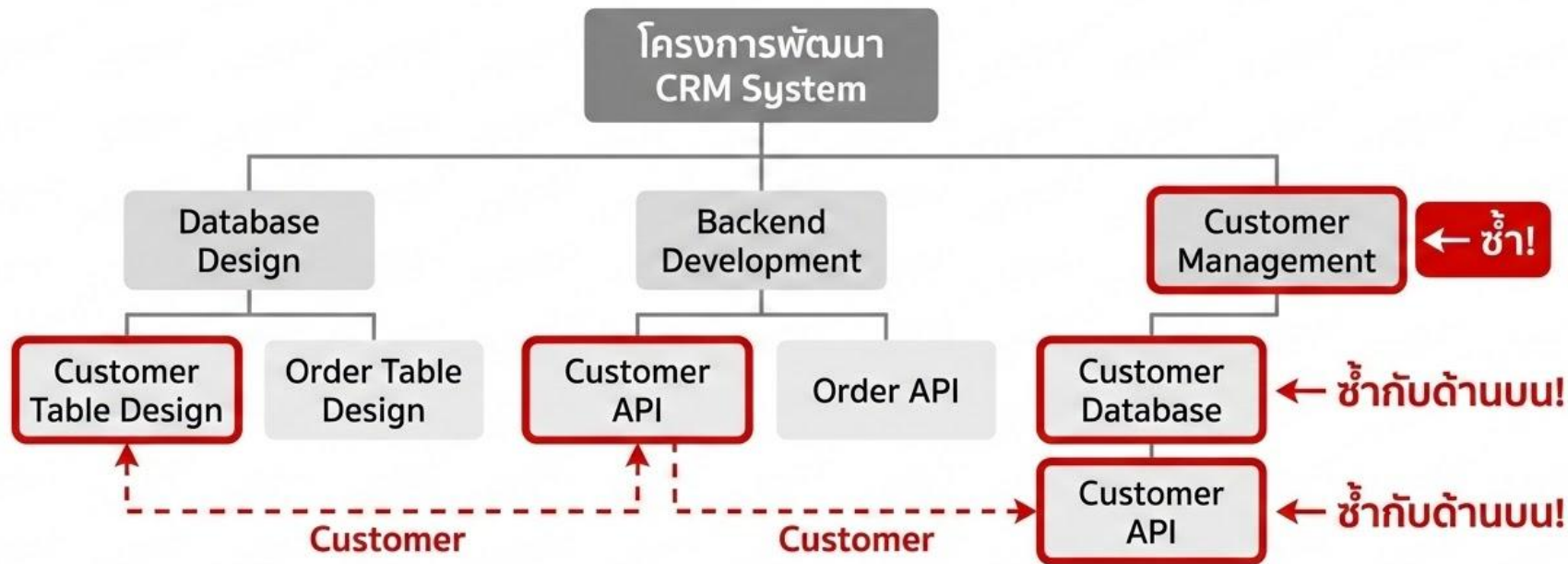
แก้ไข #2: แบ่งย่อยให้ชัดเจน



ชัดเจน ประเมินการได้ มอบหมายได้

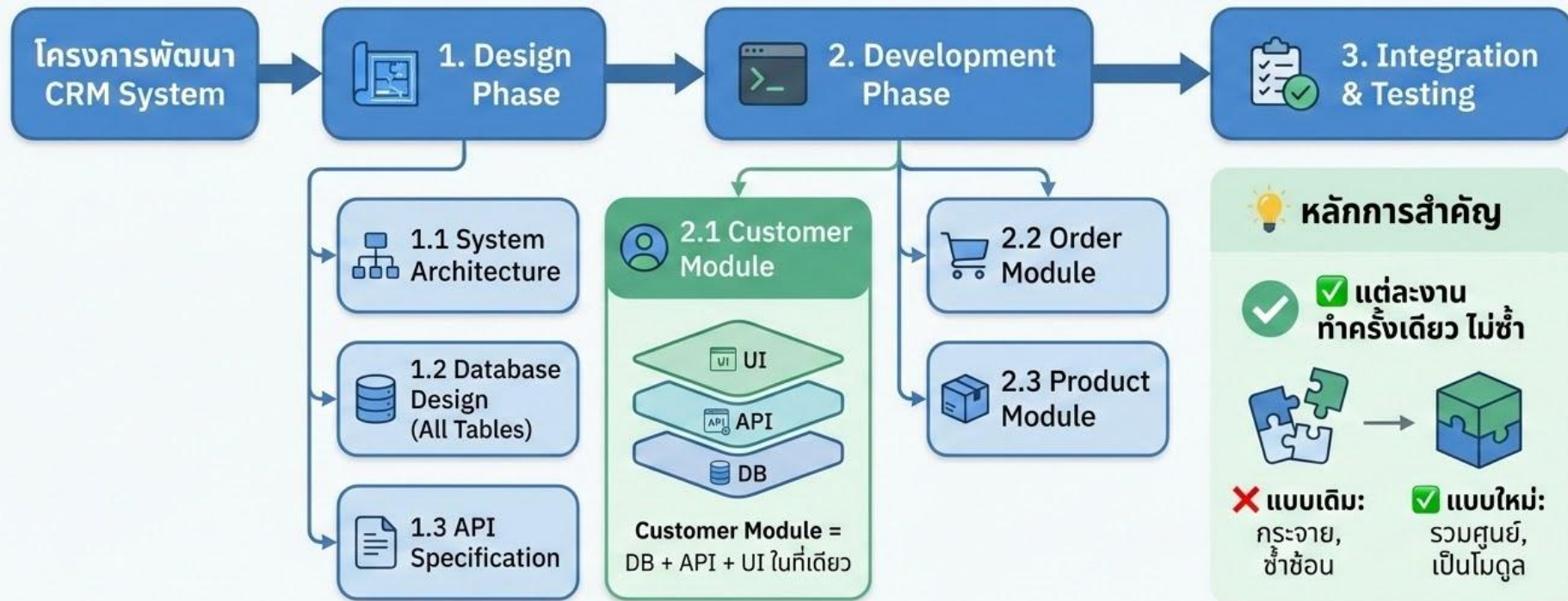
✗ WBS ที่ไม่ดี #3: ซ้ำซ้อน

ปัญหา: งานทับซ้อนกัน (Not Mutually Exclusive)



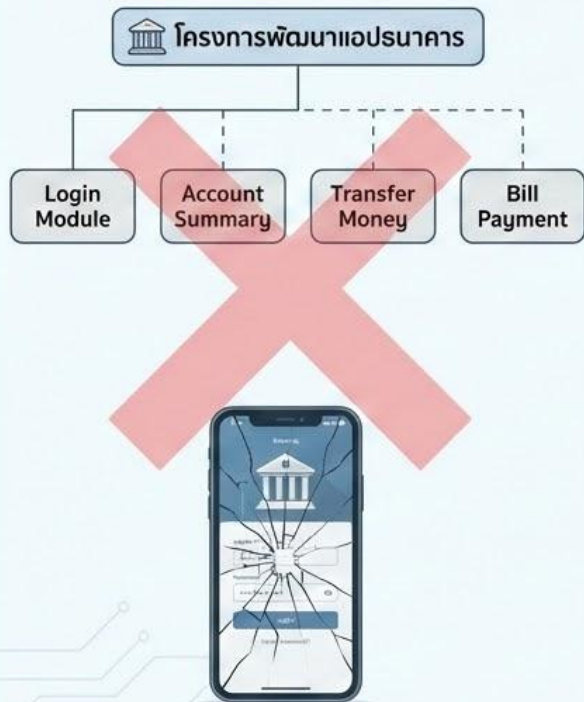
✗ งาน Customer ซ้ำกันถึง 3 ที่!

✅ แก๊โซ่ #3: แยกงานไม่ซ้ำซ้อน



WBS ที่ไม่ดี #4: ไม่ครบถ้วน

❌ WBS ที่ไม่ดี (Incomplete)



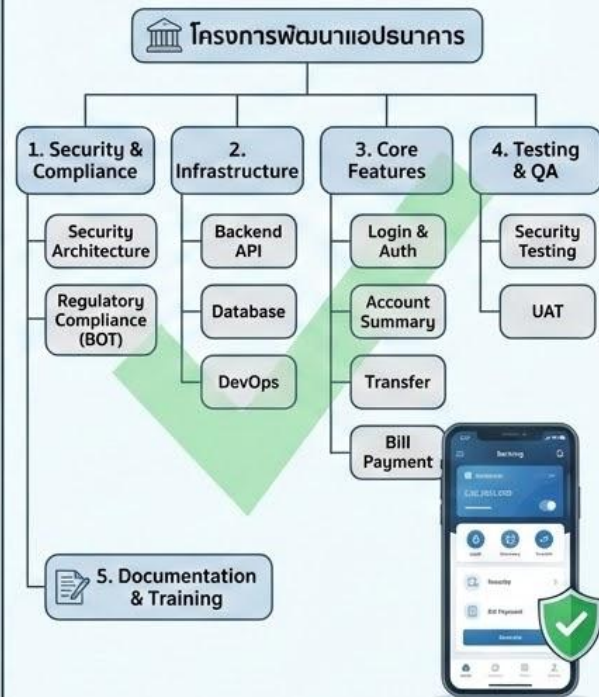
🚫 สิ่งที่ลืม (Missing Elements)

- ❌ Security & Authentication (สำคัญมาก!)
- ❌ Testing & QA
- ❌ Documentation
- ❌ DevOps & Deployment
- ❌ Backend Infrastructure
- ❌ Database Design
- ❌ Compliance & Regulations



ถ้าทำตาม WBS นี้
→ แอปไม่สมบูรณ์!

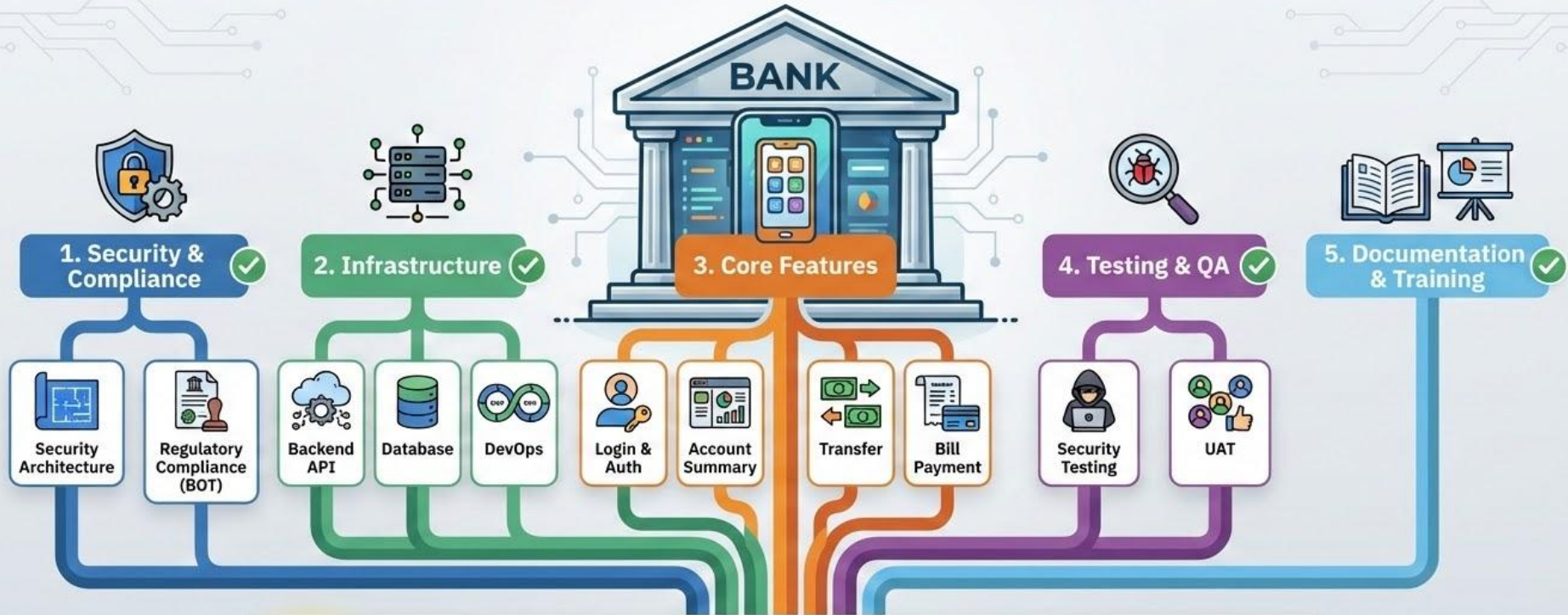
✅ แก้ไข #4: เพิ่มให้ครบถ้วน (Corrected & Complete)



✅ ครบทุกมิติ รวมถึงงานที่ไม่ใช่ Feature

✓ แก๊ซ #4: เพิ่มให้ครบถ้วน

โครงการพัฒนาแอปธนาคาร



✓ ครบทุกมิติ รวมถึงงานที่ไม่ใช่ Feature

❌ WBS ที่ไม่ดี #5: ตั้งชื่อไม่ชัดเจน

โครงการพัฒนา LMS



❌ ปัญหา:



- ไม่รู้ว่าต้องทำอะไร



- ไม่มีคำกริยาชัดเจน



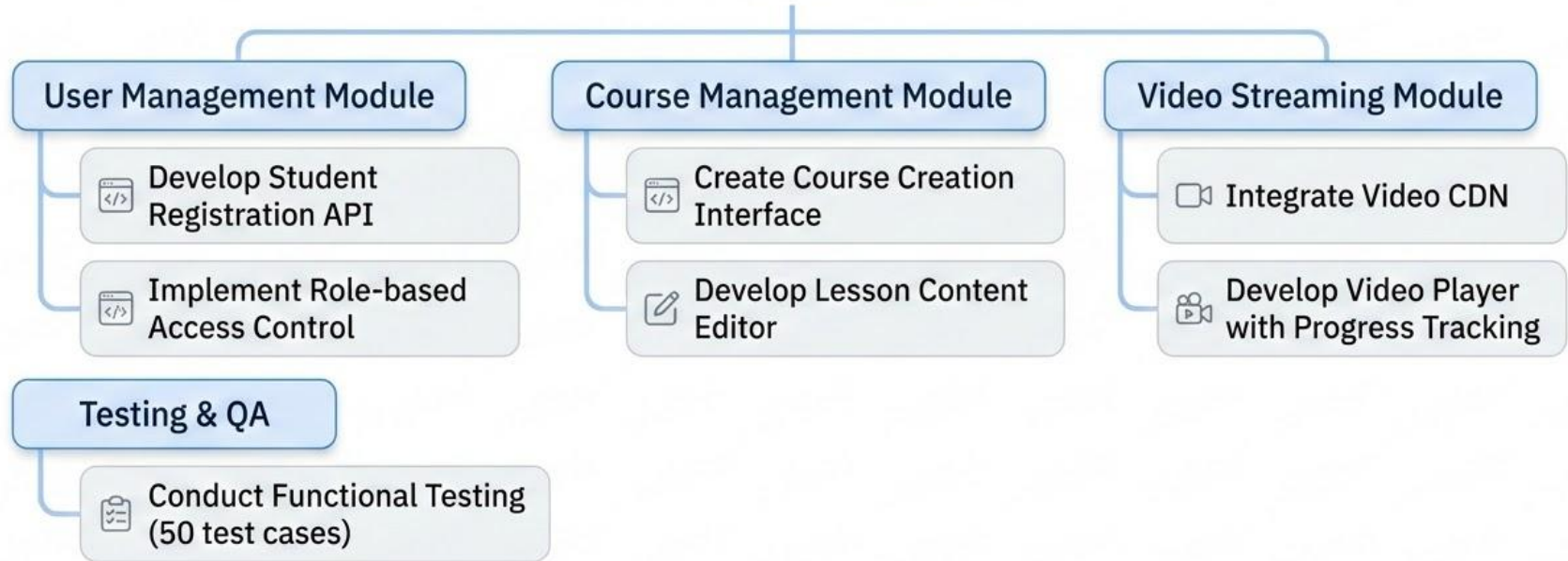
- Deliverable ไม่ชัดเจน



- ไม่สามารถมอบหมายงานได้

✅ แก๊ไข #5: ตั้งชื่อชัดเจน

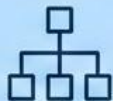
โครงการพัฒนา LMS



✅ ชัดเจน: กริยา + คำนาม + ผลลัพธ์ที่วัดได้

WBS Checklist - ตรวจสอบใช้งาน

โครงสร้าง:



-  มี 3-5 Levels
-  แต่ละงาน 8-80 ชม.
-  แบ่ง Level สมดุล

ความครบถ้วน:



-  ครอบคลุมงาน 100%
-  มี Testing/QA
-  มี Documentation
-  มี Deployment

ความชัดเจน:



-  ชื่อมี กริยา+คำนาม
-  มี Deliverable
วัดผลได้
-  ไม่มีงานซ้ำซ้อน



ผ่านทุกข้อ = WBS ที่ดี!

PART 5
COMPLETE WBS
COMPONENTS

WBS ที่สมบูรณ์ต้องมีกลุ่มงานอะไรบ้าง?



1 Project Setup & Infrastructure



2 Requirements & Planning



3 Design



4 Development (Features)



5 Testing & QA



6 Data & Content



7 Deployment & Go-Live



8 Training & Documentation



9 Project Management



นักศึกษาฝึกงาน: #1, #6, #8 → ทำให้โครงการไม่สมบูรณ์!

1 Project Setup & Infrastructure

“ถ้าระบบไม่มีอะไรเลย เราต้องเตรียมอะไรก่อน?”



1.1 Development Environment Setup

- Dev Server (Local/Cloud)
- Version Control (Git, GitHub/GitLab)
- IDE & Tools (VS Code, IntelliJ)
- Team Access & Permissions



1.2 Development Tools & Libraries

- Frontend Framework (React, Vue)
- Backend Framework (Express, Django)
- Package Managers (npm, pip)



1.3 Database Environment

- Database Server Installation & Config



1.4 DevOps & CI/CD Pipeline

- Jenkins, GitHub Actions



ประมาณ: 40-60 ชม.

ตัวอย่างรายละเอียด - Environment Setup

1.1.3 IDE & Tools Installation (8 ชม.)



Frontend Developer

- ✓ VS Code + Extensions
- ✓ Node.js (v18+) + npm
- ✓ React DevTools



Backend Developer

- ✓ IntelliJ IDEA / PyCharm
- ✓ Java JDK 17+ / Python 3.10+
- ✓ Postman (API Testing)



Mobile Developer

- ✓ Android Studio + SDK
- ✓ Xcode (for iOS)
- ✓ Emulators/ Simulators



DevOps

- ✓ Docker + Docker Compose
- ✓ Kubernetes CLI (kubectl)
- ✓ Terraform



Deliverable:

Environment Setup Checklist → ทุกคนพร้อมเริ่มงาน

6 Data & Content Preparation

ถ้าระบบไม่มีอะไรเลย เราต้องเตรียมอะไรก่อน?

6.1 Master Data Preparation

- รายการสินค้า (Product Catalog)
- ข้อมูลผู้ใช้เริ่มต้น (Seed Users)
- ข้อมูลอ้างอิง (Reference Data)
 - ประเทศ, จังหวัด, อำเภอ
- Static Content (Terms, Privacy Policy)

6.2 Data Migration (ถ้ามีระบบเก่า)

- Data Extraction from Legacy System
- Data Cleaning & Transformation
- Data Import & Validation

6.3 Test Data Preparation

- Unit Test Data Sets
- Performance Test Data (Large Dataset)

6.4 Media Assets

- Images (Products, Banners, Icons)
- Videos (Tutorials, Demos)

 ประมาณ: 50-80 ชม. |  มักถูกมองข้าม แต่สำคัญมาก\!

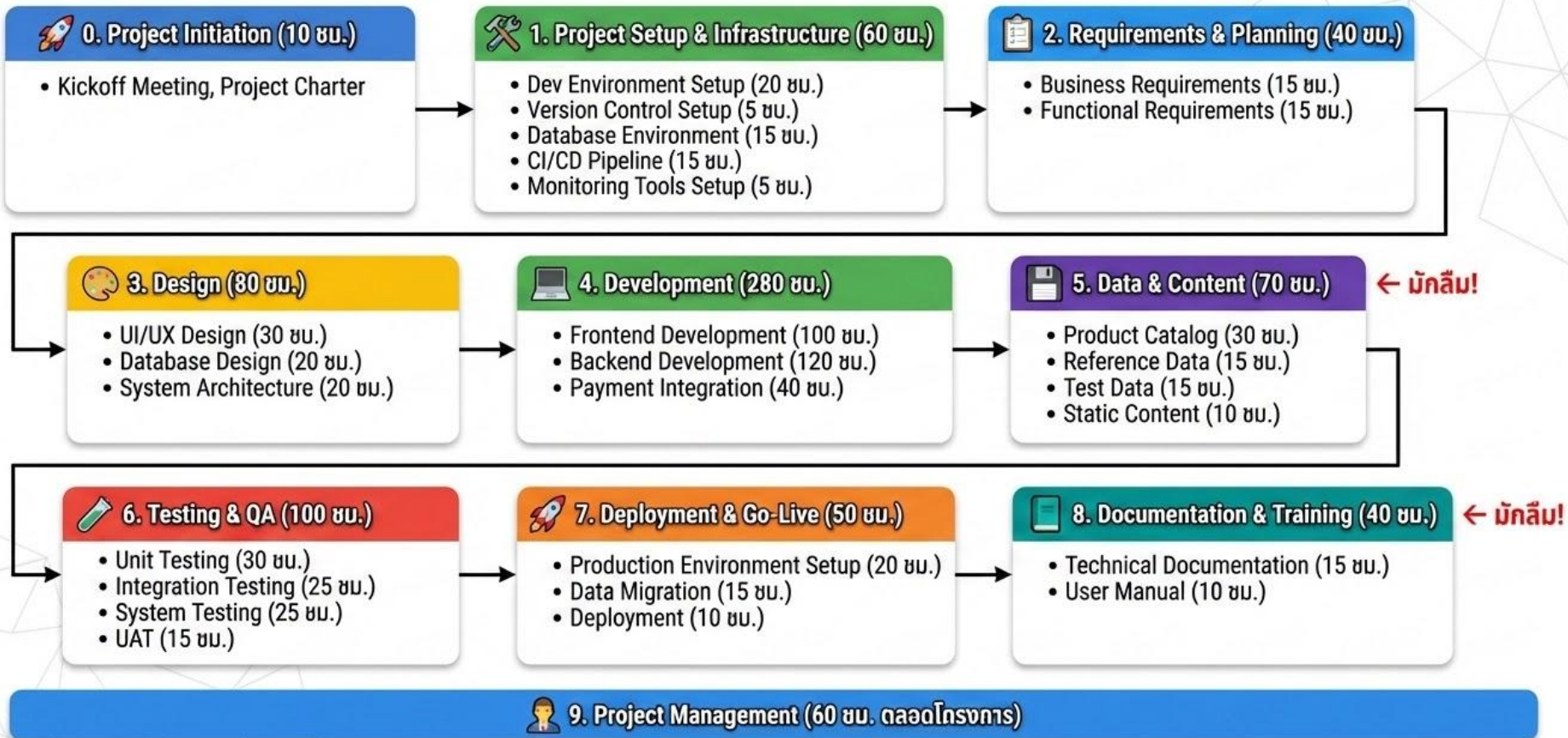
ตัวอย่างรายละเอียด - Data Preparation

6.1.1 Product Catalog Preparation (20 ชม.)

ตัวอย่าง: ระบบ E-Commerce ขายอุปกรณ์อิเล็กทรอนิกส์

- ❖ 1. รวบรวมข้อมูลจาก Vendor (10 ชม.) 
- 2. ถ่ายภาพสินค้า / Resize (5 ชม.)
- 3. เขียน Description / Review (3 ชม.)
- 4. Import เข้า Database (2 ชม.) 

WBS ครบถ้วน - E-Commerce (รวมหน้า 95-97)



งานที่มักถูกลืมไว้ใน WBS



1. Environment & Tools Setup

- ✓ IDE, Libraries, Git, CI/CD, Database Server



2. Data Preparation

- ✓ Master Data, Reference Data, Test Data, Content



3. Third-party Integration Setup

- ✓ Payment Gateway, SMS/Email Service, Cloud Storage



4. Security & Compliance

- ✓ SSL Certificate, Encryption, PDPA Compliance, Backup



5. Documentation

- ✓ API Docs, User Manual, Admin Guide, Deployment Guide



6. Training

- ✓ Admin Training, User Training

PART 6
SUMMARY & BEST
PRACTICES

เปรียบเทียบ WBS ดี vs ไม่ดี

✓ WBS ที่ดี (Good WBS)

- ✓ มีโครงสร้างชัดเจน (Hierarchical Structure)
- ✓ ครอบคลุมงานทั้งหมด (100% Rule)
- ✓ แต่ละงาน 8-80 ชม. (8/80 Rule)
- ✓ ไม่มีงานซ้ำซ้อน (Mutually Exclusive)
- ✓ ตั้งชื่อชัดเจน วัดผลได้ (Clear, Measurable Names)
- ✓ รวมงาน Non-development (Includes Testing, Docs, etc.)

✗ WBS ที่ไม่ดี (Bad WBS)

- ✗ ละเอียดยหรือหยาบเกินไป (Too Detailed or Too Vague)
- ✗ งานไม่ครบถ้วน (Missing Tasks)
- ✗ งานซ้ำซ้อนกัน (Overlapping Tasks)
- ✗ ตั้งชื่อคลุมเครือ (Vague Names, e.g., 'Stuff')
- ✗ ลืมงานสำคัญ (Forgets Testing, QA, Deployment)
- ✗ ไม่สมดุล (Unbalanced Structure)

💡 WBS ที่ดีคือรากฐานของโครงการที่ประสบความสำเร็จ

Top 7 WBS Mistakes

-  ลืมงาน **Non-development** (Testing, Documentation, DevOps, Security)
-  งานหายหรือละเอียดเกินไป (ไม่ตรงกับ **8/80 Rule**)
-  ตั้งชื่อไม่ชัดเจน ("Stuff", "Things", "Fix bugs")
-  งานซ้ำซ้อน (ทำงานเดียวกันหลายที่)
-  WBS ไม่สมดุล (บาง branch ละเอียด บาง branch หยาบ)
-  ไม่มี **Deliverable** ชัดเจน (ไม่สามารถวัดความสำเร็จได้)
-  ไม่ทบทวนและปรับปรุง (สร้างครั้งเดียวไม่แก้ไข)

💡 วิธีสร้าง WBS ที่ดี - 5 ขั้นตอน





คำถามให้คิด - Data Requirements

ด้าน Environment:

- ☐ Developer ต้องติดตั้งอะไร?
- ☐ Server ต้อง Spec อะไร?
- ☐ Database ใช้อะไร?
- ☐ Cloud Service ใช้อะไร?

ด้าน Data:

- ☐ ต้องการข้อมูลอะไรตั้งแต่วันแรก?
- ☐ ข้อมูลมาจากไหน? (Manual/Import/API)
- ☐ จำนวนข้อมูลเท่าไร?
- ☐ ใครรับผิดชอบเตรียมข้อมูล?

ด้าน Integration:

- ☐ ต้องเชื่อมต่อ Service ภายนอกอะไรบ้าง?
- ☐ ต้องสมัคร Account/API Key อะไรบ้าง?
- ☐ มีค่าใช้จ่ายหรือไม่?



ตอบได้ครบ → WBS จะครบถ้วน!

สรุป + ถัดไป

✓ สรุปสิ่งที่เรียนรู้:

- 🧠 WBS = แบ่งโครงการเป็นชิ้นเล็ก
- ✓ 3 ประเภท: Phase, Deliverable, Hybrid
- ✓ กฎ 4 ข้อ: 100%, 8/80, ไม่ซ้ำ, ครบ
- ✓ ระดับ 3-4 Levels เหมาะสม
- ✓ ตั้งชื่อให้ชัดเจน วัดผลได้

🚫 ข้อผิดพลาดที่ต้องหลีกเลี่ยง:

- ✗ งานหาย/ละเอียดยเกินไป
- ✗ ลืมงาน Testing/Docs/DevOps
- ✗ ตั้งชื่อคลุมเครือ
- ✗ งานซ้ำซ้อน/ไม่ครบ

ถัดไป: Workshop

- ➡ ลงมือสร้าง WBS แล้วหาข้อผิดพลาดกัน!

? คำถาม? 